Robotics and Its Applications

Lab Assignment - 9

Name : Naishadh Rana

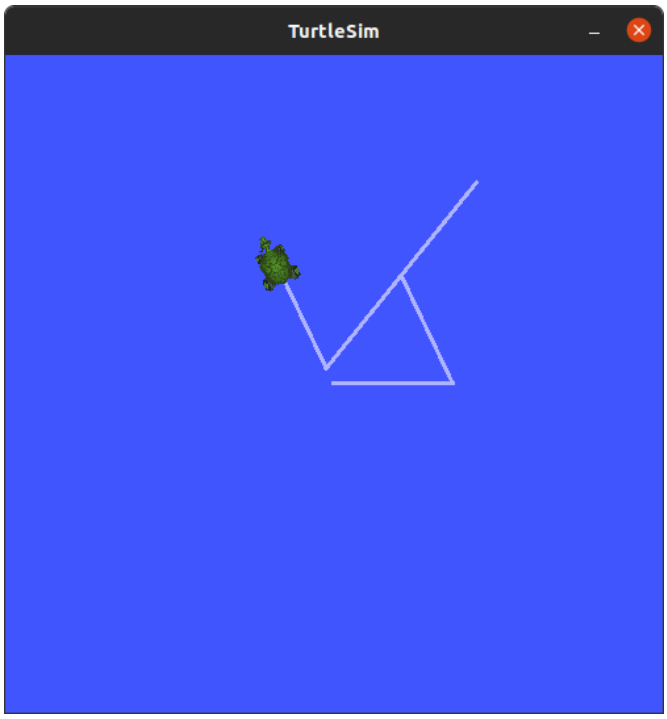
Enrollment Number : U23CS014

Part A: Turtlesim

1. Run turtlesim & teleop

```
administrator@cs-pro:~$ source /opt/ros/foxy/setup.bash
administrator@cs-pro:~$ ros2 run turtlesim turtlesim_node
[INFO] [1756884698.877128548] [turtlesim]: Starting turtlesim with node name /tu
rtlesim
[INFO] [1756884698.879554746] [turtlesim]: Spawning turtle [turtle1] at x=[5.544
445], y=[5.544445], theta=[0.000000]
administrator@cs-pro:~$ source /opt/ros/foxy/setup.bash
administrator@cs-pro:~$ ros2 run turtlesim turtlesim_node
[INFO] [1756884857.621205802] [turtlesim]: Starting turtlesim with node name /tu
rtlesim
[INFO] [1756884857.622681384] [turtlesim]: Spawning turtle [turtle1] at x=[5.544
445], y=[5.544445], theta=[0.000000]
```

```
administrator@cs-pro:~$ source /opt/ros/foxy/setup.bash
administrator@cs-pro:~$ python3 teleop.py
Control Turtle with keys:
   W = forward | S = backward
   A = left    | D = right
   Q = quit
Enter command (W/A/S/D/Q): w
Enter command (W/A/S/D/Q): a
Enter command (W/A/S/D/Q): www
Invalid key, use W/A/S/D/Q
Enter command (W/A/S/D/Q): w
Enter command (W/A/S/D/Q): a
Enter command (W/A/S/D/Q): s
Enter command (W/A/S/D/Q): w
Enter command (W/A/S/D/Q): w
Enter command (W/A/S/D/Q): d
Enter command (W/A/S/D/Q): w
Enter command (W/A/S/D/Q): q
Exiting teleop...
administrator@cs-pro:~$
```


TurtleSim

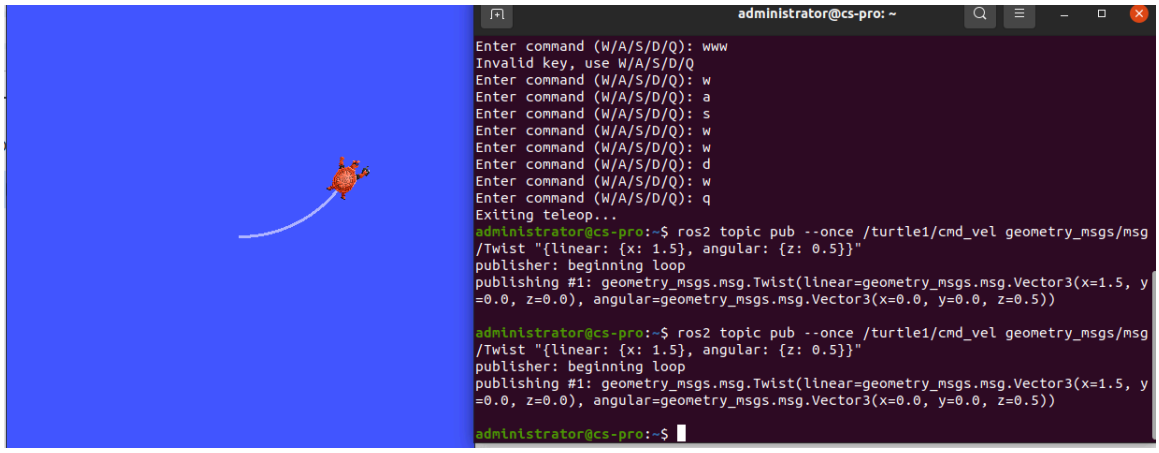
2. Explore the ROS2 Basic Commands : ros2 run, ros2 node list, ros2 topic, ros2 service, ros2 param.

```
administrator@cs-pro:~$ ros2 node list
administrator@cs-pro:~$ ros2 topic list
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

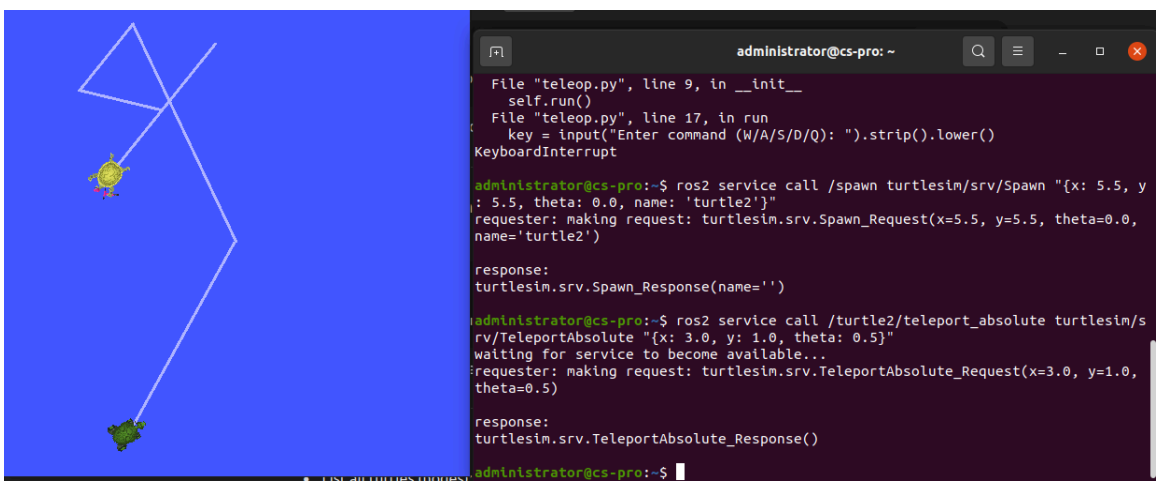
```
administrator@cs-pro:~$ ros2 service list
/clear
/kill
/reset
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/describe_parameters
/turtlesim/get_parameter_types
/turtlesim/get_parameters
/turtlesim/list_parameters
/turtlesim/set_parameters
/turtlesim/set_parameters_atomically
```

```
administrator@cs-pro:~$ ros2 param list
/turtlesim:
  background_b
  background_g
  background_r
  use_sim_time
```

3. Send a single message to the turtlesim telling it to move with a linear velocity of 1.5 and an angular velocity of 0.5. Observe the turtle’s updated motion. It will move from its starting position along a circular trajectory for a distance and then stop.

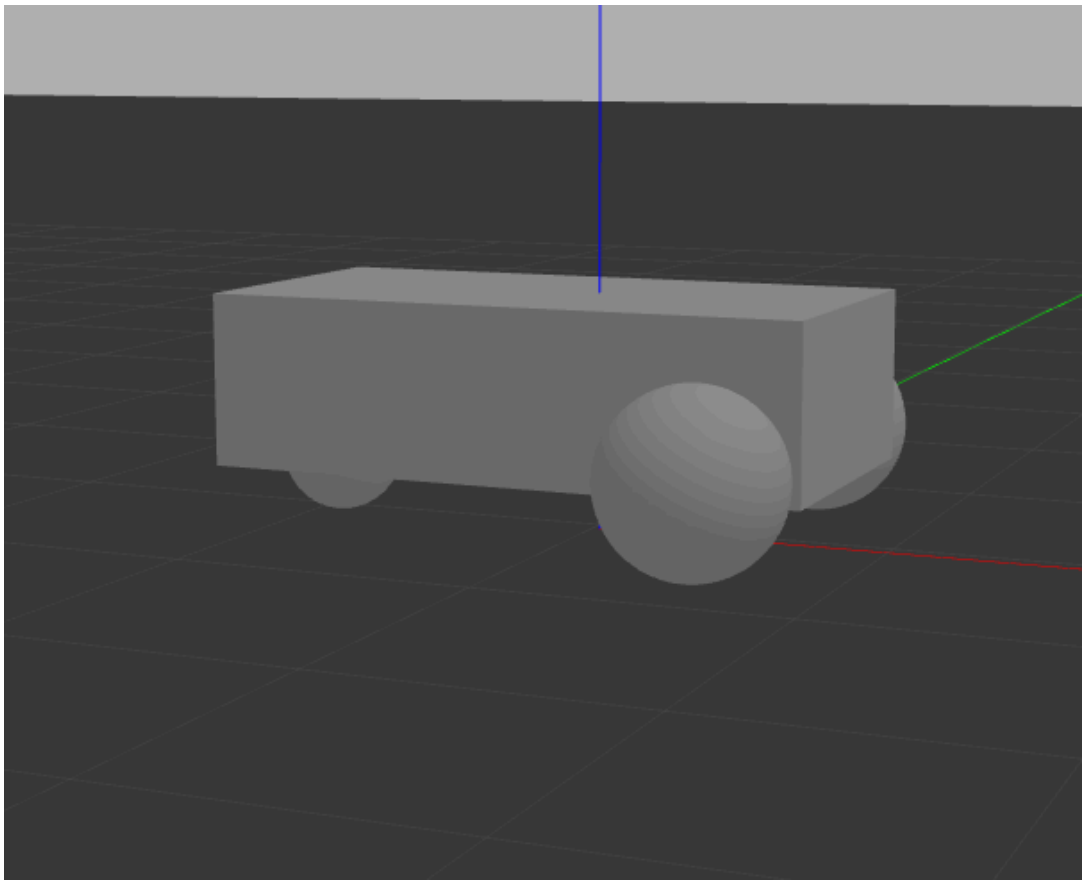


4. Without stopping the running turtlesim node, spawn a new turtle in the simulator. Using terminal commands, move this second turtle to the position (x = 3, y = 1, θ = 0.5).



Part B: Gazebo Simulation

Design and build a wheeled differential-drive robot in the Gazebo simulation environment.



Part C: Exploratory Porblems

1.1: History of ROS

The Robot Operating System (ROS) is an open-source framework used for building robot applications.

- ROS was first developed in 2007 at the Stanford Artificial Intelligence Laboratory
- Later, it was developed further by Willow Garage
- ROS 1 was widely used but had limitations in real-time and security
- ROS 2 was introduced to solve these issues and support modern robotics systems

ROS is not an actual operating system, but a middleware that helps different parts of a robot communicate.

1.2: Examples of ROS-compatible robots and hardware

Some popular robots that use ROS:

- TurtleBot3 → Used for learning and research
- PR2 Robot → Advanced research robot
- Fetch Robot → Industrial and research use
- UR5 Robot Arm → Used in automation

Hardware used with ROS includes:

- Sensors (LiDAR, cameras)
- Motors and actuators
- Microcontrollers (Arduino, Raspberry Pi)

1.3: Robot Simulators

Robot simulators help in testing robots without real hardware.

1. Gazebo
   - Most widely used with ROS
   - Provides realistic physics simulation
   - Supports sensors and robot models
2. Webots
   - Easy to use
   - Good for education and prototyping
3. V-REP / CoppeliaSim
   - Advanced simulation features
   - Supports AI and complex robotics

2. Vision-Language-Action Models and RT-2

VLA Models

- Combine
  - Vision (images)
  - Language (commands)
  - Action (robot control)
- Help robots understand instructions and perform tasks
- Example: “pick the red object”

RT-2 (Robotic Transformer 2)

- Developed by Google DeepMind
- Uses transformer-based AI
- Can understand instructions and perform tasks without explicit programming
- Helps robots generalize tasks in real-world environments

3. Explore the TurtleBot3 in the Gazebo simulation environment. Spawn the TurtleBot3, move it forward and backward using ROS 2 commands, and observe how it responds to velocity commands.

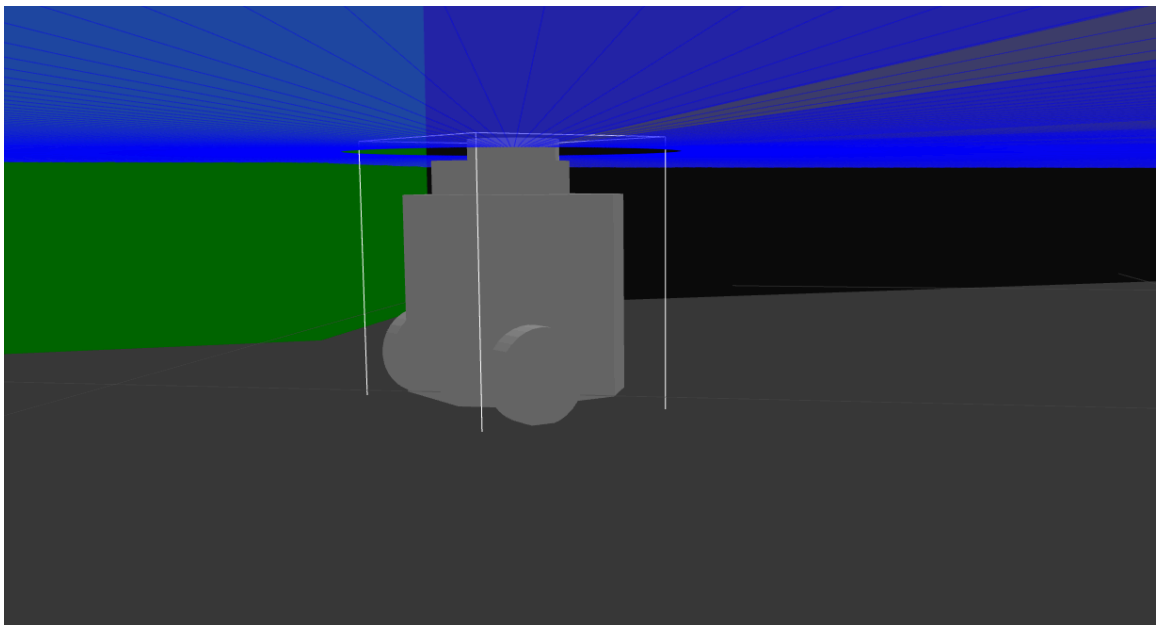
TurtleBot3 Installation:

```
sudo apt update

sudo apt install ros-foxy-turtlebot3 ros-foxy-turtlebot3-simulations
```

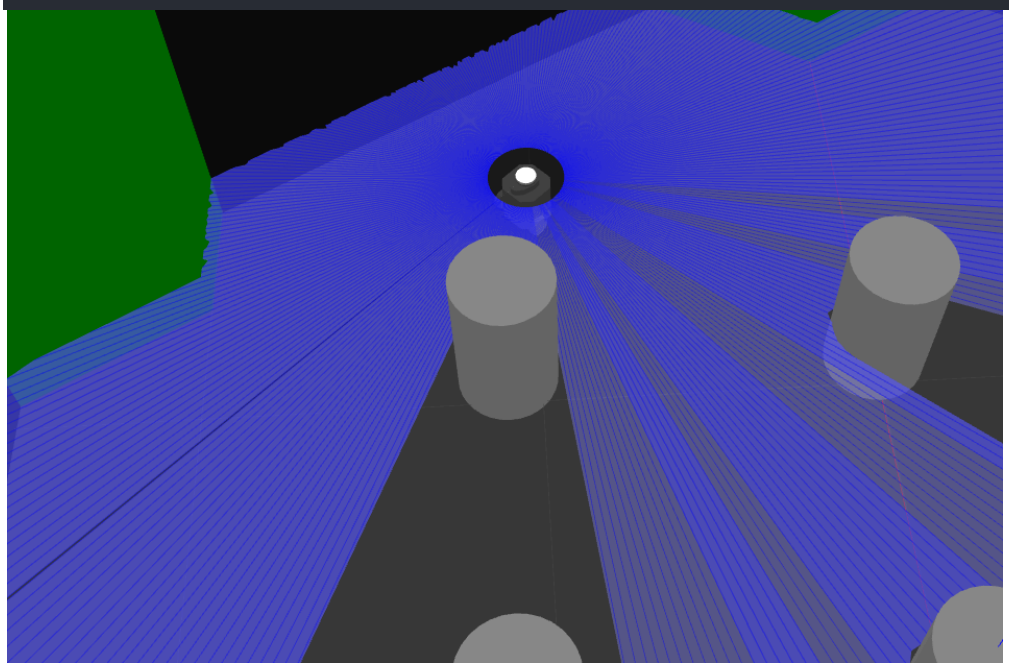
Launch TurtleBot:

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```



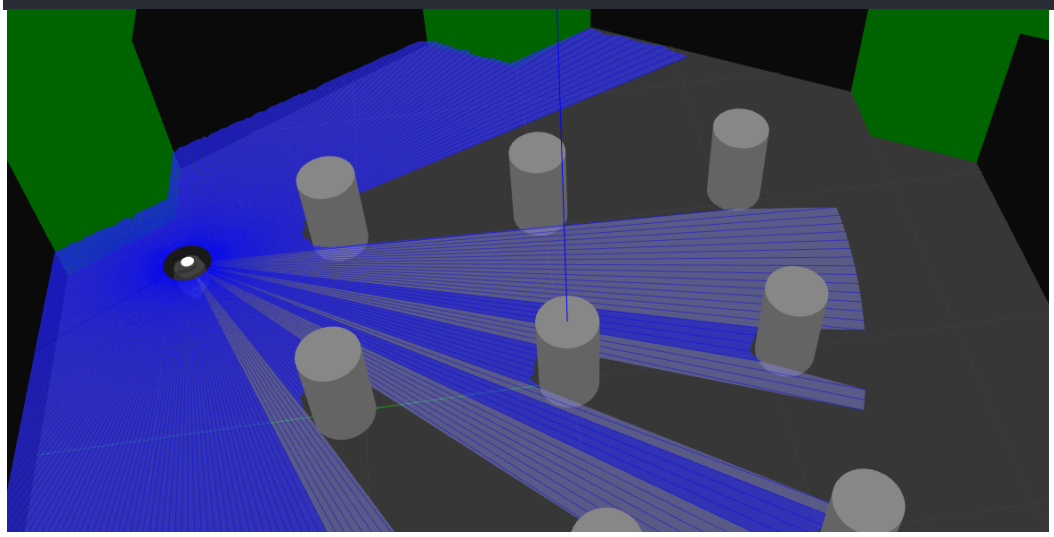
Moving it forward:

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.2, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}" -1
```



Move it backward:

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: -0.2, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}" -1
```



4. Explore how we can configure the hardware of TurtleBot3 Burger robot.

Main components:

- Raspberry Pi → runs ROS
- OpenCR board → controls motors
- LiDAR sensor → detects obstacles
- Motors and wheels → movement
- Battery → power supply

Configuration steps:

1. Assemble robot
2. Connect sensors and motors
3. Install ROS on Raspberry Pi
4. Upload firmware to OpenCR
5. Connect all components
6. Run robot using ROS